

Lab 4: Animation

王唐欣宇 — 2400017808

November 12, 2025

1 Task 1: Inverse Kinematics

1.1 Forward Kinematics

I implemented forward kinematics by recursively calculating the position of each joint based on its parent's position and its own rotation angles. Transformation matrices were used to apply rotations and translations to compute the final position of the end effector.

1.2 CCD Inverse Kinematics

I implemented the Cyclic Coordinate Descent (CCD) algorithm for inverse kinematics. The algorithm iteratively adjusts each joint angle to minimize the distance between the end effector and the target position. The process continues until the end effector is sufficiently close to the target or a maximum number of iterations is reached.

1.3 FABRIK Inverse Kinematics

I implemented the Forward And Backward Reaching Inverse Kinematics (FABRIK) algorithm. This method involves two main phases: a forward reaching phase where the end effector is moved towards the target, and a backward reaching phase where the joints are adjusted to maintain the lengths of the segments. This process is repeated until convergence.

1.4 Custom Trajectory Drawing using IK

I drew my name "WTXY" using the implemented inverse kinematics algorithms. I defined a series of target positions that trace the letters of my name and used the IK algorithms to move the end effector along these positions.

A boolean value `useEinstein` is used to switch between drawing my name and drawing Einstein.

The results of different trajectory patterns are shown in Figure 1.

1.4.1 Bonus 4.1: Uniform Sampling

We implemented an adjustable sampling function. If two neighboring points are too far apart, we subdivide the segment and insert more points between them. Note that the curve is not continuous, so we set a maximum subdivision level to avoid infinite loops. The hyperparameter `epsilon` is set to 1×10^{-3} , `max_subdivisions` is 4, and 10 points are inserted at each subdivision step. We reduced the initial sampling to 1000 points, and the final point count is around 5620, which does not exceed the original 5000-point sampling budget by too much. With this approach, we obtained smooth curves on Einstein's hair.

The comparison between uniform and adjustable sampling is shown in Figure 2.

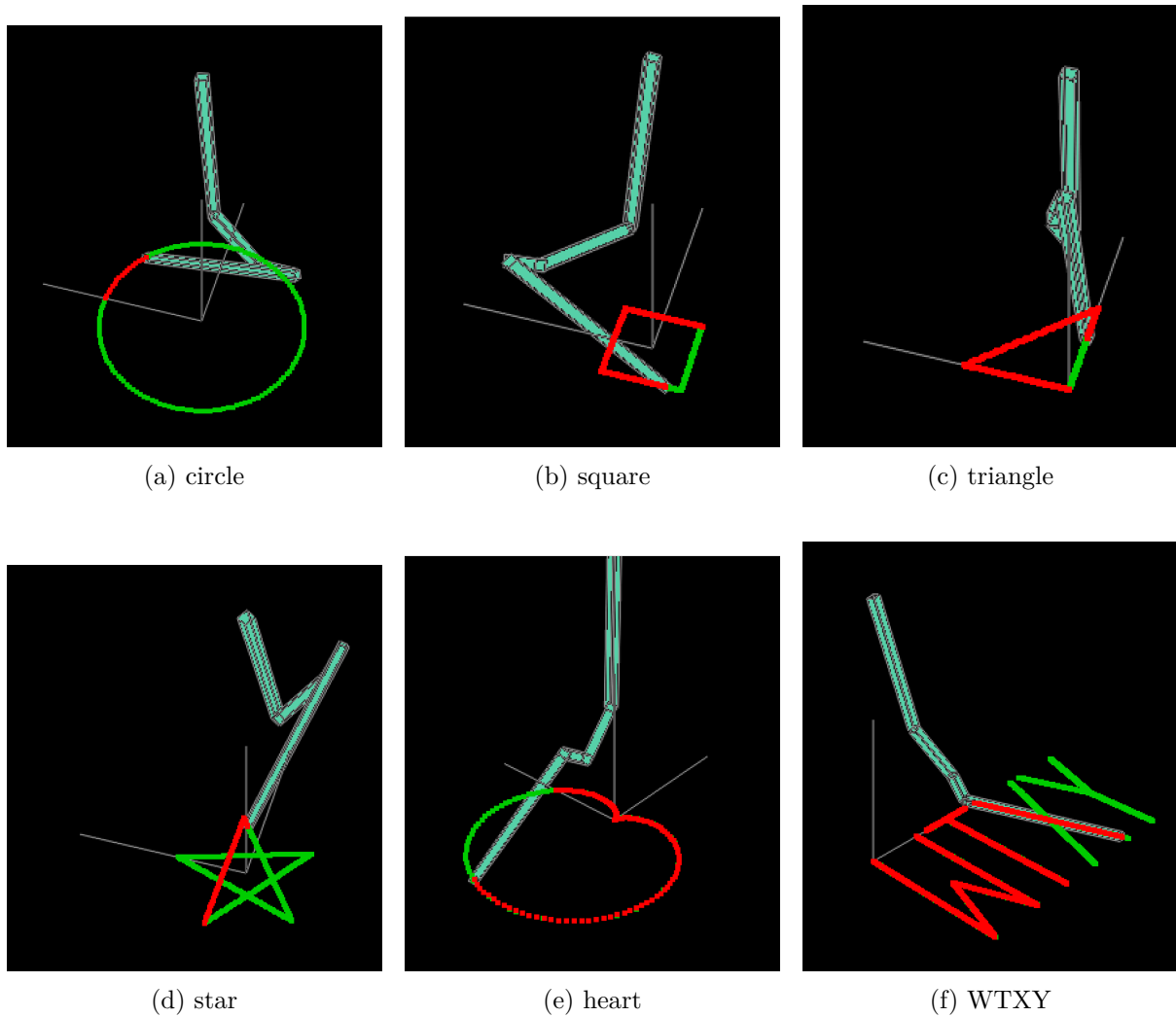


Figure 1: The Results of Task 1.

1.5 Answers to Questions

When the target is out of reach The IK algorithm will try to get as close as possible to the target, but it will not be able to reach it. The end effector will stop at the maximum extension of the arm, with all joints straightened out in the direction of the target.

Comparison between CCD and FABRIK I recorded the average number of iterations required for both algorithms to reach the target from random positions. I found that CCD has a lower convergence rate than FABRIK in my implementation, and in most cases it cannot even reach the target with a fairly large tolerance such as $\varepsilon = 10^{-3}$.

The following results are from 2600 test points with a maximum of 5000 iterations and $\varepsilon = 10^{-3}$.

As shown in Table 1, FABRIK achieves much better results than CCD in both convergence rate and average iterations.

2 Task 2: Mass-Spring System

I implemented Implicit Euler Integration following the instructions from the lectures.

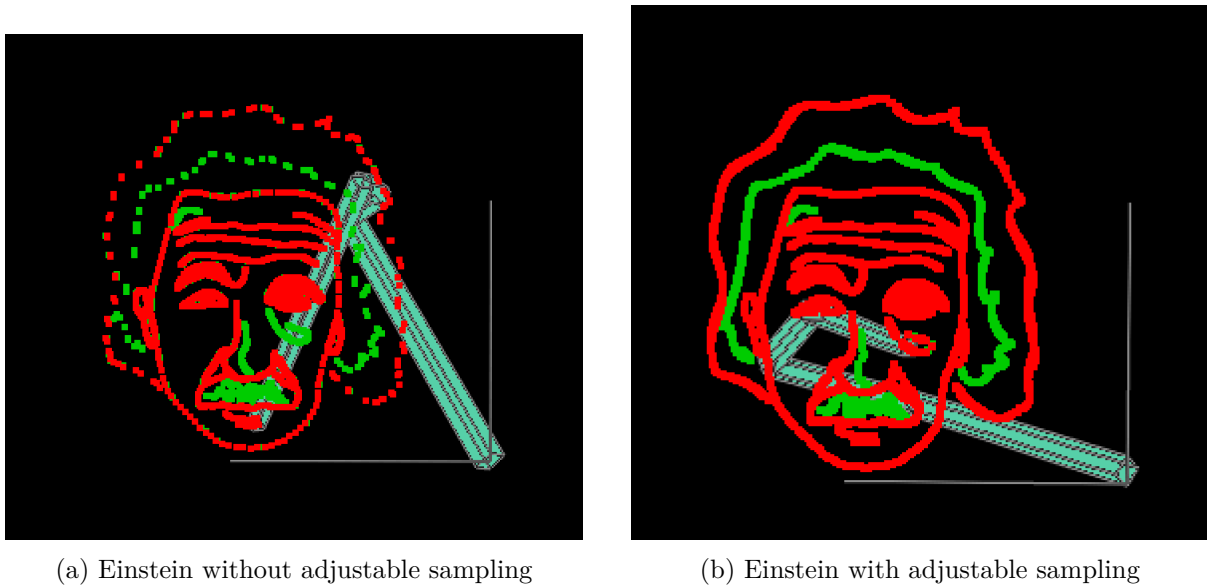


Figure 2: The Results of Bonus 1.

Table 1: Convergence Statistics for IK Algorithms

Method	Converged	Missed	Avg Iterations
CCD	206	2394	3.95
FABRIK	2590	10	1.18
CCD/FABRIK Ratio		3.361	

One challenge was how to fix the two anchor points. I used a clever approach: first, I set the mass of the fixed points to 10^8 and excluded them from gravity calculations. At the end of each iteration, I reset the fixed point positions to their original locations, which effectively keeps them stationary. This simple adjustment method works well without introducing complex constraints.

The results are quite good, and we can achieve satisfactory simulation with fewer substeps. With the substep count set to 2, the simulation runs at 24 FPS on my computer. The simulation result is shown in Figure 3.

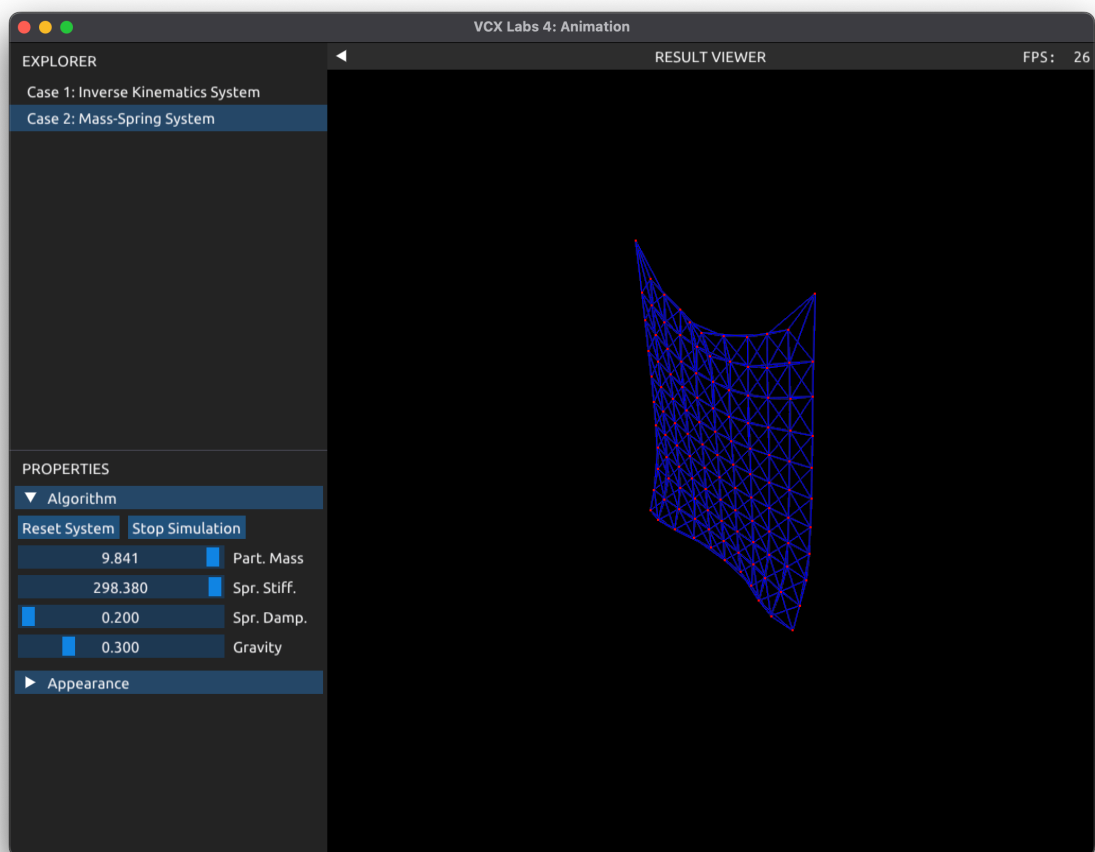


Figure 3: The Result of Task 2: Mass-Spring System Simulation.